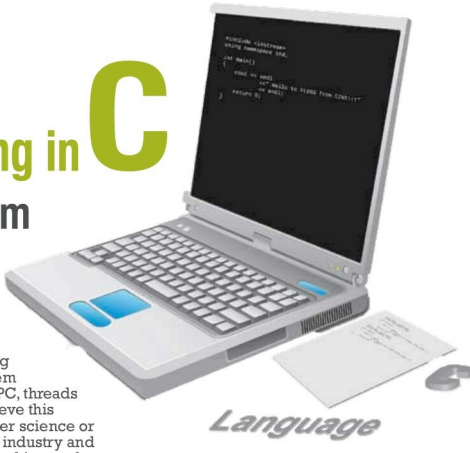
System Programming in C

The File-system Interface



In this series of articles, let us explore some important systems programming concepts in Linux, like file-system interaction, process concepts, IPC, threads and socket programming. I believe this series will be useful for computer science or IT students, freshers joining the industry and for professionals who have done this on other systems. I assume that readers are familiar with the C language and the GCC compiler on Linux, and have some basic general OS knowledge.

Part—1

In this article, we will look at some important file-system interaction-related concepts in Linux. Two types of interactions are very common—accessing or manipulating file contents through an editor, for instance; and accessing or manipulating file attributes, like listing files with the *ls* command. So here's an overview of inodes, hard and soft links, and some system call interfaces for files and directories, related to the two aspects mentioned. I will also relate these concepts to C standard library functions, so that you get an idea of how functions like *fopen*, *fclose*, *fgets*, etc, are implemented by library vendors for a particular OS.

Files, directories, inodes and links

Every file or directory in Linux has a unique number called the *inode number*. To view it, use *ls –li* (*l* is long listing format; *i* is for the inode number) at the command prompt. The first column lists the inode number of the file/directory.

The inode number is an index to a very large array of *inode structures*. Individual elements store the file's attributes (like the type of file, access permissions, the owner, number of hard links, the file size, time stamps, etc) and the addresses of the file's data blocks.

Directories have entries called directory entries, which typically have a file or a directory name, and the inode number. For example, if you create a file called *file1* in your current working directory, a free inode (say *i1*) will be allocated for it. Thus, a new directory entry of the form (*file1*, *i1*) will be created in the current directory.

Hard links (we'll come to soft links soon) are references to a particular inode. Now, if we have another directory entry (say *file2*, *i1*) that points to the *same* inode *i1*, then *file2* is referred to as a *hard link* to *file1*; both entries point to the same inode structure, and refer to the